



Exercise 2

Testing and Reliability of Electronic Systems

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

Professor's remarks:

In Exercise 2 I was not pointed out any specific error because the exercise 2 was not checked, due to previous mistakes of exercise 1.

Errors occurred as a continuation of exercise 1:

- The shift-in – shift-out cycles were 9 and not 5 as they should have been for the correct implementation of exercise 2.1 and observation in exercise 2.2
- It existed in simulations without waiting for the process $SE='1'$ (shift) by 1-2 cycles.
- Addition of the $Q_ff_name(TR_CUT)$ signals to the simulation, in order to correctly verify the shiftpage in the Scan Chain of the pseudo-random vectors produced by our LFSR.

Mistakes I noticed according to the general observations:

- Lack of some auxiliary signals from simulation ($SO(TRCUT)$, $sign(MISR)$, $q7(LFSR)$), but also their informative names.
- Incomplete description and verification of screenshots from the simulations we ran .

How did I fix them?

- Add the $next_vec$ just like in exercise 1 and so we normally have 4 circles $SE='1'$ and one circle $SE='0'$.
- Removal of delays from the testbenches within the loops, by deleting commands such as (wait 1 ns etc), for more details on the APPENDIX code K3 & K6.
- Necessary (or simply changing their names for ease of observation) have been added to verify signals such as $Q_ff_name(TR_CUT)$, ($SO(TRCUT)$)->I named a mark cut_to_misr , $sign(MISR)$, $q7(LFSR)$ ->I named a $lfsr_to_cut$ mark so that it would be easier for the reader to follow it in the midst of so many signals, for more details Exercise 2.2 and 2.4.
- Addition of mathematical operations for the correct verification of subsequent states or operation of the circuit, but also for substantiation of claims, for more details Exercise 2.2 and 2.4.

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

Exercise 2.1

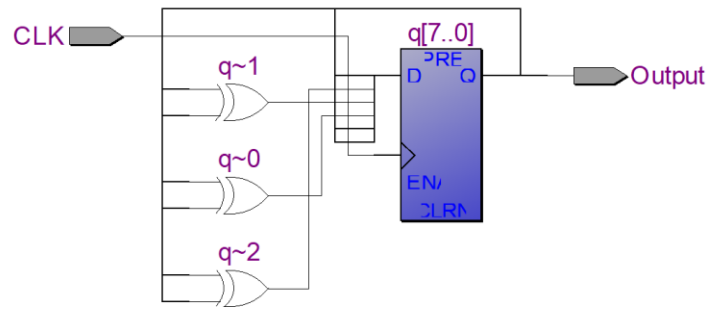
The use of **Linear Feedback Shift Registers (LFSR)** is the most widely used method for producing test vectors in **BIST (Built-In Self Test) systems**.

- **Function as a Pattern Generator (PG):** The LFSR functions as a pseudorandom vector generator. It produces a sequence of states that, although deterministic (fully defined by its structure), exhibits statistical properties similar to those of a random sequence. In our case, the 8th degree LFSR can produce up to **255 ($2^8 - 1$)** different vectors before the sequence is repeated.
- **Characteristic Polynomial $P(x)$:** The function of the LFSR is mathematically described by a polynomial. The degree of the polynomial (here $N=8$) determines the number of Flip-Flops required. The polynomial terms indicate the **feedback taps** from the Flip-Flops whose values are combined through **XOR** gates to return to the input. A primitive **polynomial** ensures the production of the maximum sequence of vectors. In this exercise we used the following polynomial **$P(x) = x^8 + x^4 + x^3 + x^2 + 1$** .
- **Importance of Seed:** The LFSR must necessarily be initialized to a non-zero state (0,0,...,0). If all Flip-Flops contain '0', the XOR operation in feedback will continue to produce '0', causing the circuit to lock-up in this state and never work.

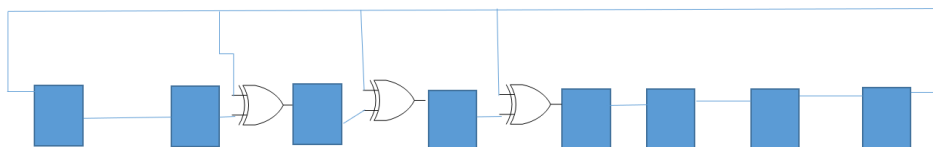
Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

RTL (LFSR8)



Power Point (LFSR8)



Code(APPENDIX K1 LFSR8_Internal.vhd)

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

Description of the Code

- **Signal q:** Represents the 8 flip-flops($P(x) = x^8 + x^4 + x^3 + x^2 + 1$). We use the value (others => '1') (i.e. 0xFF) as the **seed**, to ensure that the circuit does not start from the forbidden state of zero.
- **The Feedback Logic:** * The command $q(0) \leq q(7)$ implements the return of the MSB signal at the beginning of the chain.
- At the points where the polynomial has "taps" (at 2, 3, 4), the input of the flip-flop results from the **XOR of the output of the previous FF and the feedback bit** ($q(7)$). This corresponds exactly to the structure shown in your screenshot (e.g. between the flip-flop 'a' and 'b').
- **Synchronization:** The use of `rising_edge(CLK)` makes the circuit synchronous, as explicitly required by the voiceover.

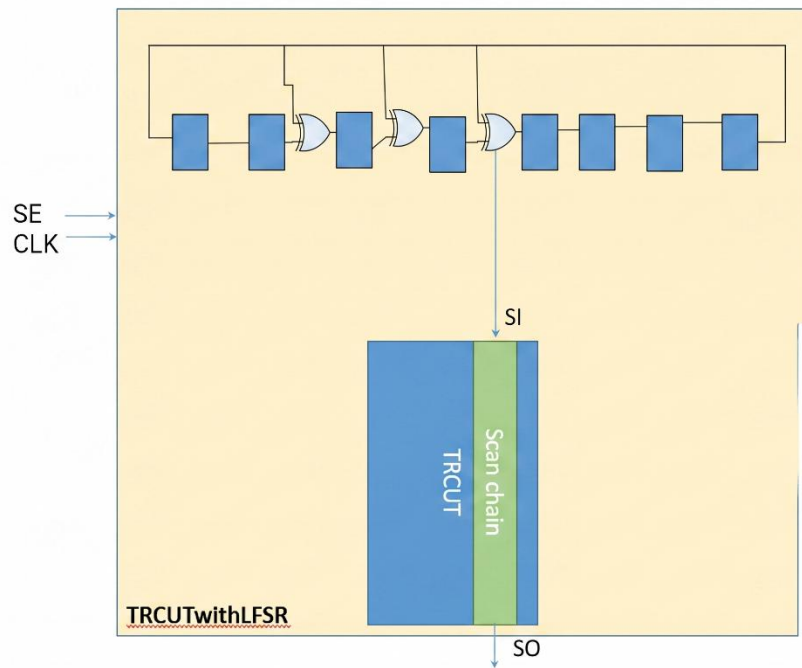
TRCUTwithLFSR Circuit Description

The **TRCUTwithLFSR** circuit is the integrated control unit for the Pseudorandom Testing phase. It is a "parent" module (top-level) that connects the vector generator to the circuit under test.

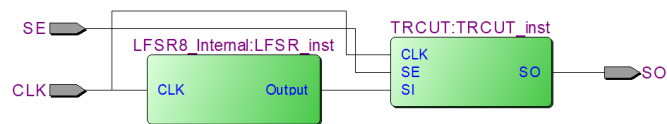
- **Structure and Composition:** The circuit consists of two main subsystems:
 - **Internal LFSR 8th degree:** as the internal vector generator (Pattern Generator). It produces a continuous stream of pseudo-random bits. According to the instructions, the **Modular (Internal)** architecture is used, which is faster due to the distribution of the XOR gates between the flip-flops.
 - **TRCUT (from Exercise 1):** It is the circuit being tested, which includes the combination logic and the scan chain (Scan Chain).
- **Wiring:** The serial output of the LFSR is connected directly to the **SI (Scan In)** input of the TRCUT. In this way, the pseudorandom bits "fill" the scan chain.
- **Control Signals:** * **CLK (Clock):** Although in the simplified form of the assignment description the clock may be omitted, the system is fully **synchronous**. The CLK signal guides both units simultaneously for the correct synchronization of the shift.
 - **SE (Scan Enable):** Controls the operation of the scan chain in TRCUT, allowing switching between shift and capture.
- **SO (Scan Out) Output:** It is the only output of the circuit, from which TRCUT's responses for analysis are serially exited.
- [Power Point \(TRCUTwithLFSR\)](#)

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381



RTL(TRCUTwithLFSR)



Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

Code(TRCUTwithLFSR APPENDIX K2)

VHDL Code Description (TRCUTwithLFSR.vhd)

The code follows the **structural modeling**, connecting the individual components that have already been implemented.

Entity

The entity defines the circuit's interface with the environment. It has:

- Two inputs: CLK (clock) and SE (scan control).
- One output: SO (scan chain output).

Architecture

Inside the architecture, the following takes place:

- **Intermediate Signal Statement:** A signal (e.g. lfsr_to_si) is used which transfers the data from the LFSR output to the TRCUT SI input.
- **Instance of LFSR (Internal):** We call the module of the LFSR 8th degree. In this code, the clock is connected to the CLK input of the generator and its output to the intermediate signal.
- **Instance of TRCUT:** We call the TRCUT module from the first exercise.
 - The **CLK** connects to the clock input of the TRCUT.
 - The SE **connects to the Scan Enable input.**
 - The intermediate signal (lfsr_to_si) is connected to the **SI input.**
 - The **SO** output of TRCUT connects directly to the external **SO output** of the module.

Operation Commentary

The code ensures that with each pulse of the clock, if the SE is active ('1'), a new pseudo-random bit moves from the LFSR to the first flip-flop of the scan chain (Rd), while at the same time the entire chain slides by one position.

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

Exercise 2.2

Testbench Function Description

The testbench simulates the BIST (Built-In Self Test) testing process by following a specific cycle for each vector:

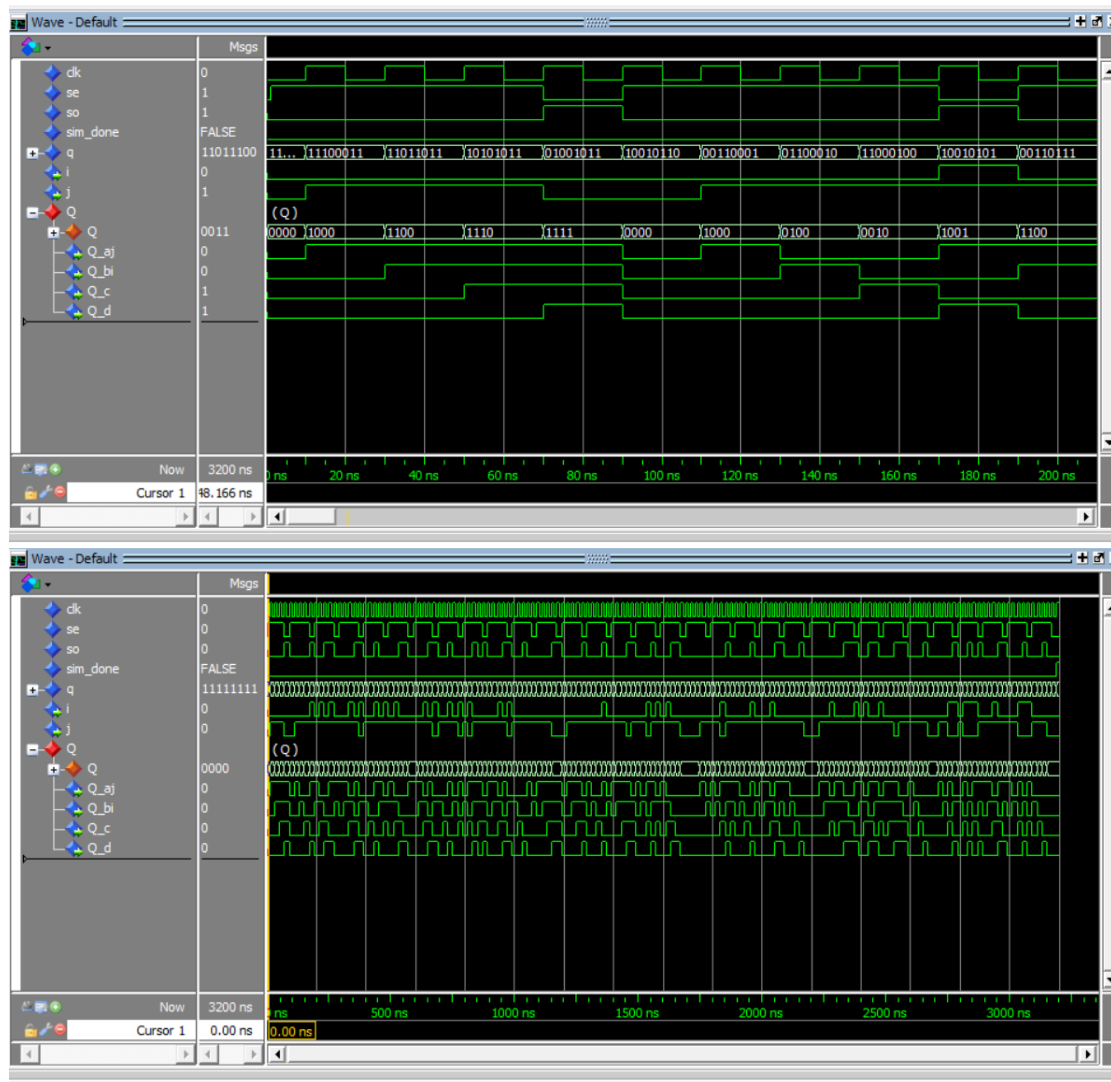
- **Shift Phase:** The **Scan Enable (SE) signal** is set to a logical '1'. For your own scan chain (consisting of 4 flip-flops: Rd, Rc, Rbi, Raj), the SE remains active for **4 clock cycles**. In each cycle, a new bit of the LFSR enters the chain, while at the same time one bit of the previous response exits the **SO**.
- **Capture Phase:** The **SE signal** is set to a logical '0' for **1 clock cycle**. During this cycle, the combined circuit (CUT) processes the values found on the flip-flops and the response (the cut_i and cut_j signals) is stored back on the flip-flops of the scan chain.
- **Repeat:** This process is repeated 32 times to apply the same number of vectors.

Code (TRCUTwithLFSR_tb.vhd APPENDIX K3)

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

Simulation(TRCUTwithLFSR_tb.vhd)



Waveform Description

1. What do you notice by collecting the answers in the testbench?

The waveform demonstrates the successful implementation of TRCUTwithLFSR through the testbench:

- Full control of the chain (4 bits shift) is ensured, through the Qs we also observe the smooth shifting of the q7 of the lfsr at q_aj->q_bi->q_c->q_d.
- The time required for the combinatorial logic to produce a result (1 clock capture) is allowed.
- The procedure is repeated for 32 vectors (32 captures of "pseudo-random" vectors), covering the requirements of Exercise 2.2.

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

- And also as we observe in the 1st photo through the logic operations of CUT

- ❖ $e \leq a \text{ xor } b$;
- ❖ $f = c \text{ xor } d$;
- ❖ $g = a \text{ xor } c$;
- ❖ $h = b \text{ xor } d$;
- ❖ $i = e \text{ and } f$; --e and f; -- Intended for RBI
- ❖ $j = g \text{ or } h$; -- Reserved for Raj

That is, for $a=1, b=1, c=1, d=1, i=0, j=0$ and in the next cycle
 $Q_{aj}=0, Q_{bi}=0, Q_c=0, Q_d=0$. The same applies to the next capture.

However, the verification of the answers on the testbench is not clear. More specifically:

2. Can you verify if the answers are correct?

Yes, it is possible but it will have to be done bit-by-bit, which makes it quite difficult to verify, because it does not necessarily follow a specific pattern (like the truth table of exercise 1), but pseudo-random values that are quite complex to calculate (I analyze this below).

3. If you can, how?

We could first write a code for predicting pseudo-randomness value through the polynomial of LFSR.

Suppose the current state (Seed) is the original of our code: "11111111" (all '1').

1. **Feedback bit (q_7):** It is '1'.

2. **Calculation (According to the polynomial of LFSR8_Internal.vhd (Appendix K1))** $P(x) = x^8 + x^4 + x^3 + x^2 + 1$.

- $q_0 = q_7 = 1$
- $q_1 = q_0 = 1$
- $q_2 = q_1 \text{ xor } q_7 = 1 \text{ xor } 1 = 0$
- $q_3 = q_2 \text{ xor } q_7 = 1 \text{ xor } 1 = 0$
- $q_4 = q_3 \text{ xor } q_7 = 1 \text{ xor } 1 = 0$
- $q_5 = q_4 = 1$
- $q_6 = q_5 = 1$
- $q_7 = q_6 = 1$

Next Status: 11100011 (Verified by the 1st screenshot of the simulation).

Then, if we have this, we have the values that will respectively pass to a, b, c, d and in the same way as with Exercise 1, we will make two variables expected_i and expected_j. But this could only apply to a circuit as small as ours! For example, in circuits such as processors where the data is billions it would be impossible to do bit-by-bit checking.

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

Exercise 2.3

MISR Analyzer Code Description (16-bit)

The MISR (Multiple Input Shift Register) acts as the Signature Analyzer. The code is based on a 16th degree LFSR with the addition of an input XOR gateway.

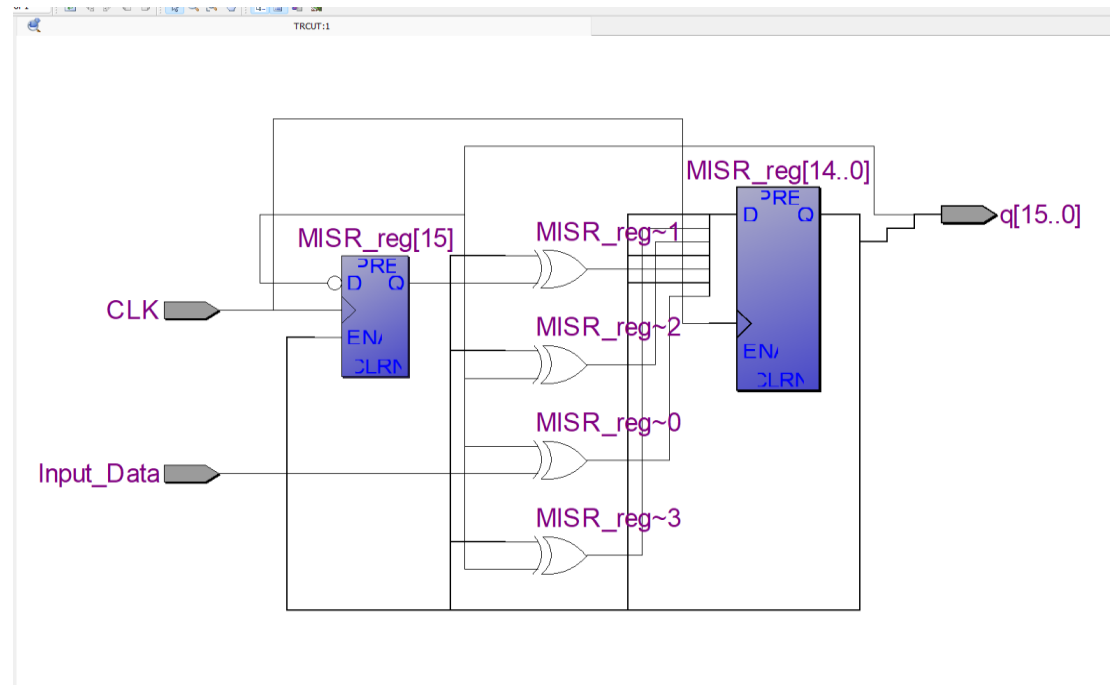
- **Register Structure:** It consists of 16 flip-flops, offering an error probability (aliasing) of only 2^{-16} , which ensures high reliability in the test.
- **Data Input:** The XOR gate at the input of the first flip-flop combines the feedback signal of the register with the **SO** response coming from the TRCUT.
- **Compaction:** In each clock cycle, the MISR "mixes" its content with the new output bit, performing a sequential compression of the responses.
- **Output Sign:** The output of the module is connected to a flip-flop of the MISR (usually the MSB), allowing the final signature to be read bit-by-bit

Code(MISR16.vhd APPENDIX K4)

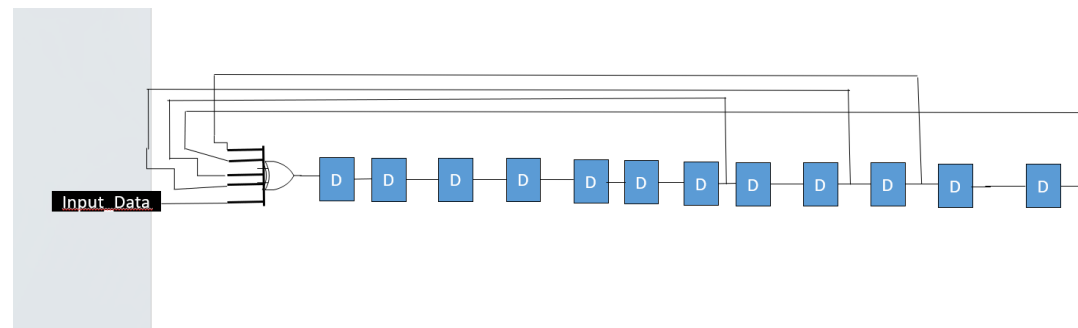
Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

RTL(MISR16)



Power Point(MISR16)



Integrated Circuit Mode (TRCUTwithMISR)

The **TRCUTwithMISR** circuit is a complete **BIST (Built-In Self Test)** architecture. Its operation is divided into three distinct phases:

A. Vector Production and Application (LFSR → TRCUT)

The LFSR generator produces pseudorandom bits. When the **SE** signal is '1' (Shift phase), these bits are serially entered into the scan chain of TRCUT (Rd → Rc → Rbi → Raj). After 4 clock beats, the chain is filled with a new test vector.

B. Response Recording and Transfer (TRCUT → MISR)

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

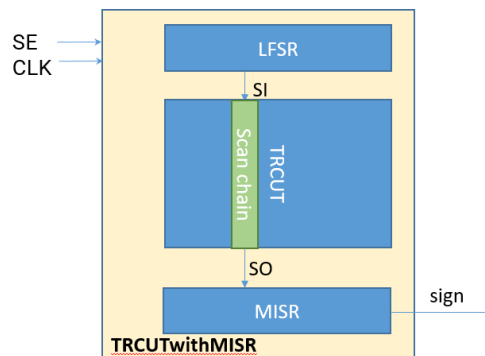
When the **SE** becomes '0' (Capture phase), the combinatorial circuit (CUT) processes the vector and the response is stored in the flip-flops of the chain. In the next shift cycle (SE=1), as the next vector enters, the previous response exits the **SO** and is led to **the MISR** for compression.

C. Signature Creation and Export (MISR Signature)

- **Compression:** After applying 32 vectors, all responses have passed through the MISR and been compressed based on the polynomial characteristic.
- **Final Signature:** The state in which the 16 MISR flip-flops are located is the only "signature" of the circuit. If the circuit has an error, the signature will be different from the expected ("correct").
- **Export:** At the end of the process, an additional 16 clock cycles are required to output the contents of the MISR serially from the sign output .

Conclusion: The system allows full control of the circuit internally, converting a sequence of 128 bits of responses (32 vectors x 4 bits) into a manageable 16-bit signature

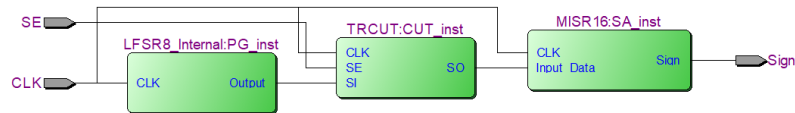
Power Point(TRCUTwithMISR)



Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

RTL(TRCUTwithMISR)



The Data Path

The connection follows a clear path:

- **LFSR → TRCUT:** The signal lfsr_out carry the control data.
- **TRCUT → MISR:** The signal so_to_misr conveys the results of the test for compression.
- **Final Result:** The Sign output of the system is the one we will compare with a known "healthy" value to tell if the chip is faulty or not.

And the correspondence of RTL with the Power Point schema given to us is fully visually verified.

Code(TRCUTwithLFSR.vhd APPENDIX K5)

Description of the code

The system consists of three entities that play the following roles:

1. **PG (Pattern Generator) – LFSR8_Internal:** It is the generator of test patterns. It uses an 8-bit LFSR to produce pseudorandom strings of '0' and '1', which will serve as inputs for the check.
2. **CUT (Circuit Under Test) – TRCUT:** This is the circuit we want to test. It receives the data from the LFSR (at the SI input) and produces some responses (at the SO

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

output). The SE (Scan Enable) signal determines whether the circuit is in normal operation or control mode.

3. **SA (Signature Analyzer) – MISR16:** It is the signature analyzer. It takes TRCUT's responses and "condenses" them into its 16-bit register. In the end, the Sign output gives us the final "**digital signature**", which is unique for the correct functioning of the circuit.

The Data Path

The connection follows a clear path:

- **LFSR → TRCUT:** The signal `lfsr_out` carry the control data.
- **TRCUT → MISR:** The signal `so_to_misr` conveys the results of the test for compression.
- **Final Result:** The Sign output of the system is the one we will compare with a known "healthy" value to tell if the chip is faulty or not.

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

Exercise 2.4

Code(TRCUTwithMISR_tb APPENDIX K6)

Description of Testbench (TRCUTwithMISR_tb.vhd)

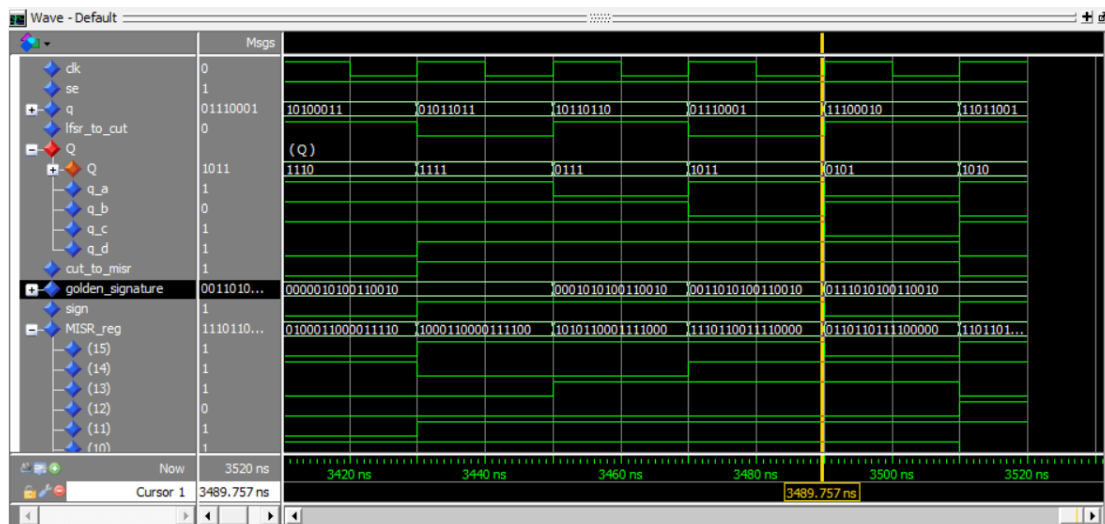
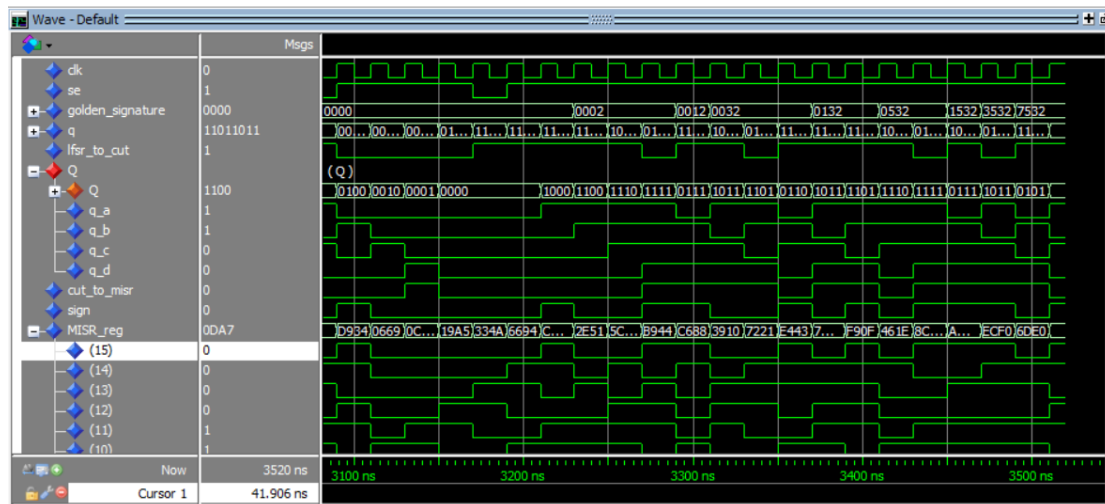
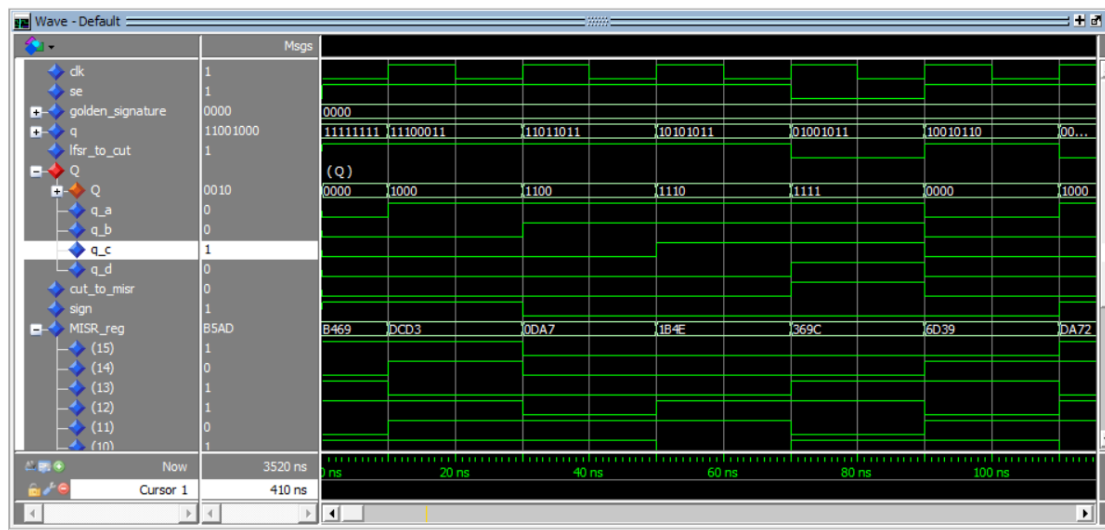
Testbench was designed to automatically test the TRCUTwithMISR circuit, simulating the control process in two distinct phases:

- **Initialization and Clock Control:** Testbench generates a periodic clock signal (CLK) with a period of 20 ns. The simulation is automatically terminated via the sim_done signal, ensuring that the clock stops immediately after collecting the 16 bits of the signature.
- **Phase A: Application of 32 pseudorandom vectors:**
 - The program runs a loop of 32 repetitions.
 - **Shift Phase (SE = '1'):** For each vector, the SE signal becomes '1' for 4 clock cycles (as much as the length of the scan chain), allowing pseudo-random data from the LFSR to enter the CUT.
 - **Capture Phase (SE = '0'):** This is followed by a clock cycle where SE becomes '0' so that the responses of the combination circuit are stored in the scan cells.
- **Phase B: Signature Extraction: After the completion** of the vectors, the Testbench keeps the SE at '1' for 16 additional cycles. In this phase, the contents of the MISR are extracted serially from the sign output and stored in the signal golden_signature.

Simulation 1 (correct)

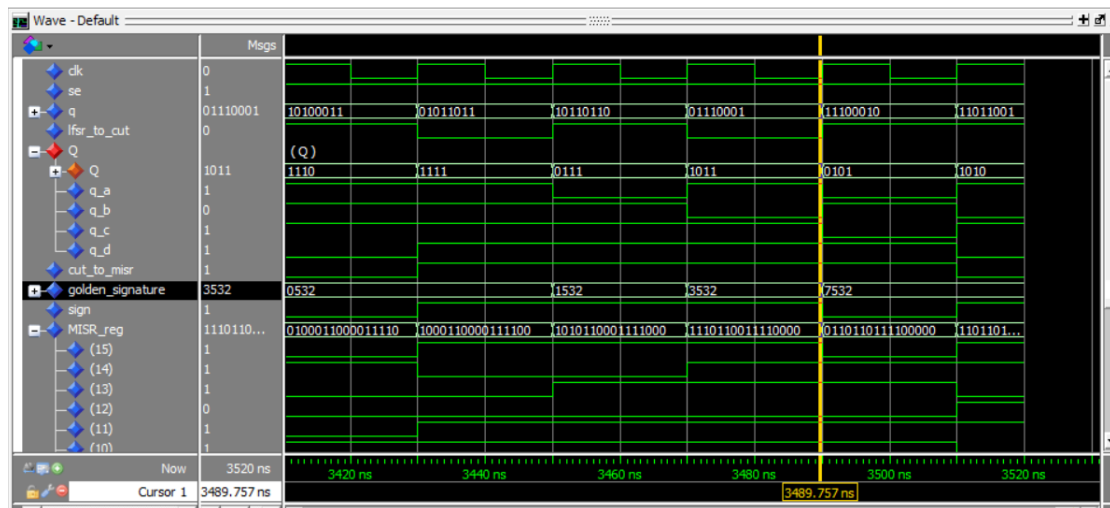
Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381



Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381



Description of Waveforms

The waveforms capture the dynamic behavior of the system and the success of the control:

- **Control Signals:** The correct switching of SE='1' (shift for 4 cycles) and SE = '0' (capture for 1 cycle) is observed 32 times as we had seen in exercise 1 and as well as the correct shifting of the LFSR data where we saw in exercise 2.2 above (2nd snapshot). We have also added the lfsr_to_cut signal for easier observation of the bits shifting from LFSR(q7) to CUT(SI).
- **Signal Stability:** After the correct initialization of the MyDFF and MISR16 registers, the sign and golden_signature signals do not show 'X' (unknown) or 'U' (uninitialized) values (1st instance).
- **Proper Implementation of Phase A:** We observe during the first phase of the testbench that the MISR works as it should based on our circuit.

Mathematical Verification of Compression

Let's consider the rising edge of the clock at **100 ns**(1st snapshot):

Pre-edge condition (t = 80 ns):

- MISR_reg = **369C** (in Hex) or 0011 0110 1001 1100 (in Binary).

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

- **Feedback** (Bit 15 of MISR): It is 0.
- **Input_Data** (from cut_to_misr): It is 1.
- **Calculating Bit 0 for the next state:**
- According to the code: MISR_reg(0) <= feedback xor Input_Data.
- Operation: 0 XOR 1 = 1.
- **Post-edge condition (t = 100 ns):**
- The simulator shows MISR_reg = **6D39** (in Hex) or 0110 1101 0011 1001 (in Binary).
- We notice that the LSB (Bit 0) of the new value is indeed **1**.

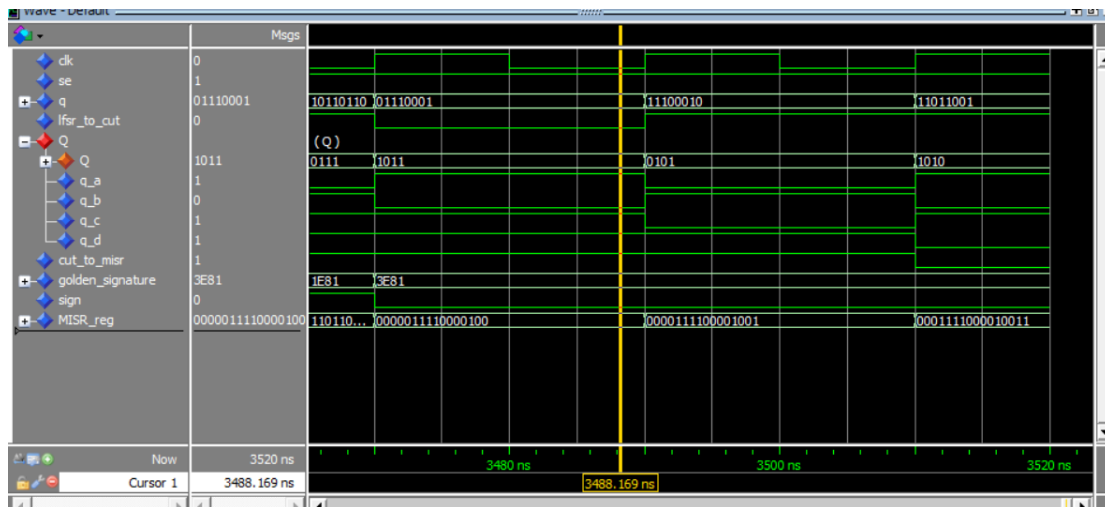
So we understand through simulation that the compression of MISR is done correctly.

- **Then** in the 2nd and 3rd snapshot we can see that the sign signal (output of MISR, MISR (15) in our snapshot) at each positive edge of the clock slides to the golden_signature and is stored at the location of the edge where the shift was made. In the first positive edge of the 16 circle slide we do let's say that sign='1', the golden_signature becomes "0000000000000001" and continues accordingly.
- **Golden Signature:** At the end of the healthy circuit simulation, the signal golden_signature receives the fixed value **of 7532**, it is worth noting that I ran the simulation several times without changing the signature (more than 20). This value is the digital identity of the correct operation of the circuit for this LFSR seed.
- **Faulty Signature:** In the second simulation (in the presence of an error), the signature changes to the value **3E81**. The visual comparison of the two waveforms shows that the output sign produced a different sequence of bits, which led to the differentiation of the signature.

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

Simulation 2(error injection)



Implementation of Fault Injection

The evaluation of the effectiveness of the test was done through the introduction of a stuck-at fault in the CUT:

- **Fault Selection:** Selected to inject a **stuck-at-1** fault into the i output of the combo circuit.
- **RTL modification:** In the CUT.vhd file, the normal logical assignment $i \leq e$ and $f?$ was replaced by the fixed value $i \leq '1'$ (**stuck at 1**);.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity CUT is
5  port (
6      a, b, c, d : in std_logic;
7      i, j       : out std_logic
8  );
9  end entity;
10
11 architecture rtl of CUT is
12     signal e, f, g, h : std_logic;
13 begin
14     -- Λογική βασισμένη στο σχήμα της Άσκησης 1 [cite: 1]
15     e <= a xor b;
16     f <= c xor d;
17     g <= a xor c;
18     h <= b xor d;
19
20     i <= '1';           -- e and f; -- Προορίζεται για το Rbi
21     j <= g or h;        -- Προορίζεται για το Raj
22 end architecture;
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

- **Testing Process:** The circuit was recompiled and simulated with the exact same Testbench and the same seed, so that the only variable is the existence of the error.
- **Detection:** Generation of a different signature (**3E81** instead of **7532**) demonstrated that the error was propagated through the scan chain to the MISR and successfully detected by the 32 pseudorandom vectors

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

APPENDIX

K1 LFSR8_Internal.vhd

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
entity LFSR8_Internal is
```

```
    port (
```

```
        CLK : in std_logic;
```

```
        Output : out std_logic
```

```
    );
```

```
end entity;
```

```
architecture rtl of LFSR8_Internal is
```

```
    -- Non-zero seed initialization (all '1')
```

```
    signal q : std_logic_vector(7 downto 0) := (others => '1');
```

```
begin
```

```
    process(CLK)
```

```
    begin
```

```
        if rising_edge(CLK) then
```

```
            -- Bit 0 always accepts feedback from bit 7
```

```
            q(0) <= q(7);
```

```
            -- Bits without XOR (single shift)
```

```
            q(1) <= q(0);
```

```
            q(5) <= q(4);
```

```
            q(6) <= q(5);
```

```
            q(7) <= q(6);
```

```
            -- Bits with XOR in between (Taps based on polynomial)
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

```
-- q(i)_next = q(i-1) XOR feedback(q7)

q(2) <= q(1) xor q(7); -- Tap on x^2

q(3) <= q(2) xor q(7); -- Tap on x^3

q(4) <= q(3) xor q(7); -- Tap on x^4

end if;

end process;

-- The output that will go to TRCUT's SI

Output <= q(7);

end architecture;
```

K2 TRCUTwithLFSR.vhd

```
library ieee;

use ieee.std_logic_1164.all;

entity TRCUTwithLFSR is

    port (

        SE : in  std_logic; -- Scan Enable

        CLK : in  std_logic; -- Clock

        SO : out std_logic -- Scan Out

    );

end entity;

architecture struct of TRCUTwithLFSR is

    signal lfsr_to_si : std_logic; -- LFSR -> TRCUT Connection Signal

begin

    -- 1. LFSR instance (8th degree)

    LFSR_inst: entity work. LFSR8_Internal

        port map (

            CLK  => CLK,
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

```
Output => lfsr_to_si
```

```
);
```

```
-- 2. Instance of TRCUT (from Exercise 1)
```

```
TRCUT_inst: entity work. TRCUT
```

```
port map (
```

```
    CLK => CLK, -- Common clock (synchronous circuit)
```

```
    SI => lfsr_to_si, -- Scan input comes from LFSR
```

```
    SE => SE,
```

```
    SO => SO
```

```
);
```

```
end architecture;
```

K3 TRCUTwithLFSR_tb

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.numeric_std.all;
```

```
use std.textio.all; -- For prints in Transcript
```

```
entity TRCUTwithLFSR_tb is
```

```
end entity;
```

```
architecture behavior of TRCUTwithLFSR_tb is
```

```
    signal clk    : std_logic := '0';
```

```
    signal se     : std_logic := '0';
```

```
    signal so     : std_logic;
```

```
    signal sim_done : boolean := false; -- Signal to end the simulation
```

```
    constant clk_period : time := 20 ns;
```

```
begin
```


Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

-- Instance of the circuit

uut: entity work. TRCUTwithLFSR

port map (

CLK => clk,

SE => se,

SO => so

);

-- Clock production that stops when sim_done = true

clk_process : process

begin

while not sim_done loop

clk <= '0'; wait for clk_period/2;

clk <= '1'; wait for clk_period/2;

end loop;

wait;

end process;

-- Main control process for 32 workloads

stim_proc: process

variable i, j : integer;

begin

-- Initialization

se <= '0';

wait for clk_period * 2;

report "--- STARTING 32 PSEUDORANDOM TEST VECTORS ---";

for i in 1 to 32 loop

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

```
-- Shift Phase: SE = '1' for 4 cycles (chain length: Rd, Rc, Rbi, Raj)

se <= '1';

for j in 1 to 4 loop

    wait until rising_edge(clk);

    -- Here you can observe the SO (Scan Out)

end loop;


-- Capture Phase: SE = '0' for 1 cycle

se <= '0';

wait until rising_edge(clk);


report "Vector " & integer'image(i) & " applied and captured.";

end loop;


report "--- 32 VECTORS COMPLETED ---";

sim_done <= true; -- The clock stops

wait;

end process;

end architecture;
```

K4 MISR16

```
library ieee;

use ieee.std_logic_1164.all;

entity MISR16 is

    port (

        CLK      : in std_logic;

        Input_Data : in std_logic; -- The SO signal from TRCUT

        q        : out std_logic_vector(15 downto 0)

    );
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

end entity;

architecture RTL of MISR16 is

-- Non-zero seed initialization

signal MISR_reg : std_logic_vector(15 downto 0) := "1011010001101001";

begin

process(CLK)

variable feedback : std_logic;

begin

if rising_edge(CLK) then

feedback := MISR_reg(15);

-- Galois MISR implementation based on the polynomial of LFSR16_1B401

-- Add XOR tap for SO input to bit 0

MISR_reg(0) <= feedback xor Input_Data;

MISR_reg(1) <= MISR_reg(0);

MISR_reg(2) <= MISR_reg(1);

MISR_reg(3) <= MISR_reg(2);

MISR_reg(4) <= MISR_reg(3);

MISR_reg(5) <= MISR_reg(4);

MISR_reg(6) <= MISR_reg(5);

MISR_reg(7) <= MISR_reg(6);

MISR_reg(8) <= MISR_reg(7);

MISR_reg(9) <= MISR_reg(8);

MISR_reg(10) <= MISR_reg(9) xor feedback;

MISR_reg(11) <= MISR_reg(10);

MISR_reg(12) <= MISR_reg(11) xor feedback;

MISR_reg(13) <= MISR_reg(12) xor feedback;

MISR_reg(14) <= MISR_reg(13);

MISR_reg(15) <= MISR_reg(14) xor feedback;

end if;

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

```
end process;  
  
q <= MISR_reg;  
  
end architecture;
```

K5 TRCUTwithMISR

```
library ieee;  
  
use ieee.std_logic_1164.all;  
  
entity TRCUTwithMISR is  
  
    port (  
  
        CLK : in std_logic;  
  
        SE : in std_logic;  
  
        sign : out std_logic -- The only signature output  
  
    );  
  
end entity;  
  
architecture struct of TRCUTwithMISR is  
  
    signal lfsr_to_cut : std_logic;  
  
    signal cut_to_misr : std_logic;  
  
    signal misr_q : std_logic_vector(15 downto 0);  
  
begin  
  
    -- 1. Connecting LFSR8 (Pattern Generator)  
  
    -- Attention: The name formal port must be 'Output'  
    -- to match the code you sent.  
  
    PG_inst: entity work. LFSR8  
  
    port map (  
  
        CLK => CLK,  
  
        Output => lfsr_to_cut -- SI Connection  
  
    );-- 2. Connecting TRCUT (Circuit Under Test)  
  
    CUT_inst: entity work. TRCUT  
  
    port map (  
  
        CLK => CLK,
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

```
SE => SE,  
  
SI => lfsr_to_cut,  
  
SO => cut_to_misr -- SO Connection  
  
);
```

-- 3. Connecting the MISR16 (Signature Analyzer)

SA_inst: entity work. MISR16

```
port map (  
  
    CLK    => CLK,  
  
    Input_Data => cut_to_misr,  
  
    q      => misr_q  
  
);
```

-- Output sign: Selecting the MISR bit 15

```
sign <= misr_q(15);
```

```
end architecture;
```

K6 TRCUTwithMISR_tb

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.numeric_std.all;
```

```
use std.textio.all;
```

```
entity TRCUTwithMISR_tb is
```

```
end entity;
```

```
architecture behavior of TRCUTwithMISR_tb is
```

```
-- Control signals (as in 2.2)
```

```
signal clk    : std_logic := '0';
```

```
signal se     : std_logic := '0';
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

```
signal sign : std_logic;
```

```
signal sim_done : boolean := false;
```

```
-- Signal to store the Golden Signature (visible in waveform)
```

```
signal golden_signature : std_logic_vector(15 downto 0) := (others => '0');
```

```
constant clk_period : time := 20 ns;
```

```
constant SCAN_CHAIN_LENGTH : integer := 4; -- Chain Length (Rd, Rc, Rbi, Raj)
```

```
begin
```

```
-- 1. Instance of the TRCUTwithMISR circuit
```

```
uut: entity work. TRCUTwithMISR
```

```
port map (
```

```
    CLK => clk,
```

```
    SE => se,
```

```
    sign => sign
```

```
);
```

```
-- 2. Clock Generation (ends with sim_done)
```

```
clk_process : process
```

```
begin
```

```
    while not sim_done loop
```

```
        clk <= '0'; wait for clk_period/2;
```

```
        clk <= '1'; wait for clk_period/2;
```

```
    end loop;
```

```
    wait;
```

```
end process;
```

```
-- 3. Main Control Process (Stimulus)
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

```
stim_proc: process

begin

    -- Initialization

    se <= '0';

    wait for clk_period * 2;

    report "--- PHASE A: APPLYING 32 TEST VECTORS ---";

    for i in 1 to 32 loop

        -- Phase Shift: SE = '1' for 4 cycles

        se <= '1';

        for j in 1 to SCAN_CHAIN_LENGTH loop

            wait until rising_edge(clk);

        end loop;

        -- Capture Phase: SE = '0' for 1 cycle

        se <= '0';

        wait until rising_edge(clk);

        -- Transcript progress report

        if (i mod 8 = 0) then

            report "Processed " & integer'image(i) & " vectors...";

        end if;

    end loop;

    report "--- PHASE B: EXTRACTING 16-BIT SIGNATURE ---";

    -- Signature Extraction: SE = '1' for 16 cycles

    se <= '1';
```

Testing and Reliability of Electronic Systems

Athanasios Fytilis, 5381

```
for i in 0 to 15 loop
    wait until rising_edge(clk);

    -- Saving the bit to the corresponding location of the signal
    golden_signature(i) <= sign;
end loop;

report "--- SUCCESS: GOLDEN SIGNATURE COLLECTED ---";

-- Shutdown to stop the simulation right here
sim_done <= true;
wait;
end process;

end architecture;
```